

CSS @import Rule

Complete Import StyleSheet Guide

Load external stylesheets and organize CSS efficiently

Easy English • 40+ Examples • Ready to Use

Contents

1	Introduction to @import Rule	2
1.1	What is @import?	2
1.2	Basic Syntax	2
1.3	Why Use @import?	2
1.4	@import vs <link> Tag	3
2	Basic Usage of @import	4
2.1	1. Importing Local Files	4
2.2	2. Importing External URLs	4
2.3	3. Different Import Syntax	5
2.4	4. Import Order Matters	5
3	Advanced @import Features	6
3.1	1. Conditional Imports with Media Queries	6
3.2	2. Import with Supports Queries	6
3.3	3. Layered Imports	6
4	Practical CSS Organization	8
4.1	Example 1: Modular CSS Architecture	8
4.2	Example 2: Theme System with @import	10
4.3	Example 3: Google Fonts Organization	10
5	Best Practices	12
5.1	Practice 1: Limit Import Depth	12
5.2	Practice 2: Use @import in Preprocessors	12
5.3	Practice 3: Critical Path Optimization	13
5.4	Practice 4: Name Imports Meaningfully	13
5.5	Practice 5: Organize Import Order	13
6	Performance Considerations	15
6.1	1. HTTP Requests	15
6.2	2. Use Build Tools	15
6.3	3. Critical CSS Inlining	15
6.4	4. Font Loading Strategy	16
7	Real-World Examples	17
7.1	Example 1: Design System	17
7.2	Example 2: Multi-Brand Setup	17

7.3	Example 3: Framework Integration	18
8	Common Mistakes to Avoid	19
8.1	Mistake 1: @import Not at Start of File	19
8.2	Mistake 2: Circular Imports	19
8.3	Mistake 3: Deep Nesting	19
8.4	Mistake 4: Wrong File Paths	20
8.5	Mistake 5: Performance Ignored	20
9	Tips and Best Practices	21
9.1	Tip 1: Use CSS Preprocessors	21
9.2	Tip 2: Leverage Build Tools	21
9.3	Tip 3: Document Your Structure	21
9.4	Tip 4: Use Meaningful Naming	22
9.5	Tip 5: Monitor Performance	22
10	Summary	23
11	Practice Exercises	24
11.1	Exercise 1: Modular Architecture	24
11.2	Exercise 2: Theme System	24
11.3	Exercise 3: Font Organization	24
11.4	Exercise 4: Framework Integration	24
11.5	Exercise 5: Multi-File Project	25
11.6	Exercise 6: Performance Optimization	25

Chapter 1

Introduction to @import Rule

1.1 What is @import?

The `@import` rule in CSS allows you to import external stylesheets or other CSS files into your current stylesheet. It enables you to organize CSS code across multiple files and reuse styles across different projects. The `@import` rule must be placed at the beginning of a CSS file, before all other rules.

Think of it as: "Load another CSS file from within your CSS file."

[colback=lightblue, colframe=codeblue, coltitle=darkblue, title=Key Concept] The `@import` rule allows you to modularize CSS by splitting it into multiple files, making code more maintainable and reusable across projects.

1.2 Basic Syntax

```
@import url("path/to/stylesheet.css");  
@import url("https://example.com/styles.css");  
@import "styles.css";
```

1.3 Why Use @import?

- Organize CSS into logical modules
- Reuse stylesheets across projects
- Import fonts from external sources
- Load conditional stylesheets
- Separate concerns (layout, typography, colors)
- Make code more maintainable
- Reduce main stylesheet size
- Enable team collaboration

1.4 @import vs <link> Tag

- **@import**: CSS-based, imports from within CSS
- **<link>**: HTML-based, imports from HTML document
- Both serve similar purposes
- **<link>** is generally preferred for better performance
- **@import** is useful when importing within stylesheets

Chapter 2

Basic Usage of @import

2.1 1. Importing Local Files

Import CSS files from your project:

[colback=codegray, colframe=darkblue, title=CSS Code]

```
@import url("./reset.css");
@import url("./variables.css");
@import url("./typography.css");
@import url("./components.css");
```

File structure:

```
project/
  styles/
    reset.css
    variables.css
    typography.css
    components.css
    main.css
```

In main.css:

```
@import url("./reset.css");
@import url("./variables.css");
@import url("./typography.css");
@import url("./components.css");

/* Your main styles here */
body { margin: 0; }
```

2.2 2. Importing External URLs

Import stylesheets from external sources:

[colback=codegray, colframe=darkblue, title=CSS Code]

```
/* Google Fonts */
@import url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&
  display=swap');
```

```
/* Bootstrap Framework */
@import url('https://cdn.jsdelivr.net/npm/bootstrap@5.0.0/dist/css/bootstrap.min.css');

/* Font Awesome Icons */
@import url('https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css');
```

2.3 3. Different Import Syntax

Multiple ways to write @import:

[colback=codegray, colframe=darkblue, title=CSS Code]

```
/* Method 1: url() function */
@import url("styles.css");

/* Method 2: url() with quotes */
@import url('styles.css');

/* Method 3: without url() - quoted string */
@import "styles.css";

/* Method 4: without url() - single quotes */
@import 'styles.css';
```

2.4 4. Import Order Matters

CSS cascade follows import order:

[colback=codegray, colframe=darkblue, title=CSS Code]

```
/* Import 1: Reset and base styles first */
@import url("./reset.css");

/* Import 2: Variables and utilities */
@import url("./variables.css");

/* Import 3: Components */
@import url("./components.css");

/* Import 4: Themes (overrides previous) */
@import url("./theme.css");

/* Your main styles last (highest priority) */
body { background: white; }
```

Chapter 3

Advanced @import Features

3.1 1. Conditional Imports with Media Queries

Import stylesheets conditionally based on device:

[colback=codegray, colframe=darkblue, title=CSS Code]

```
/* Import mobile styles for small screens */
@import url("./mobile.css") (max-width: 768px);

/* Import tablet styles */
@import url("./tablet.css") (min-width: 769px) and (max-width: 1024px);

/* Import desktop styles */
@import url("./desktop.css") (min-width: 1025px);

/* Import print styles */
@import url("./print.css") print;
```

3.2 2. Import with Supports Queries

Load stylesheets based on browser features:

[colback=codegray, colframe=darkblue, title=CSS Code]

```
/* Load modern CSS Grid styles if supported */
@import url("./grid-layout.css") supports(display: grid);

/* Load flexbox styles if supported */
@import url("./flexbox.css") supports(display: flex);

/* Load CSS custom properties styles */
@import url("./css-vars.css") supports(--custom: property);
```

3.3 3. Layered Imports

Organize imports with CSS Layers:

[colback=codegray, colframe=darkblue, title=CSS Code]

```
/* Define layer order */
@layer base, theme, utilities;

/* Import styles into specific layers */
@import url("./reset.css") layer(base);
@import url("./colors.css") layer(theme);
@import url("./utilities.css") layer(utilities);

/* Utilities layer has lowest specificity, base highest */
```

Chapter 4

Practical CSS Organization

4.1 Example 1: Modular CSS Architecture

[colback=codegray, colframe=darkblue, title=File Structure]

```
css/  
  main.css (imports all)  
  base/  
    reset.css  
    typography.css  
    forms.css  
  components/  
    buttons.css  
    cards.css  
    navbar.css  
    footer.css  
  layout/  
    grid.css  
    flexbox.css  
    spacing.css  
  utilities/  
    colors.css  
    margins.css  
    animations.css
```

[colback=codegray, colframe=darkblue, title=main.css]

```
/* 1. Base styles */  
@import url("../base/reset.css");  
@import url("../base/typography.css");  
@import url("../base/forms.css");  
  
/* 2. Layout styles */  
@import url("../layout/grid.css");  
@import url("../layout/flexbox.css");  
@import url("../layout/spacing.css");  
  
/* 3. Component styles */  
@import url("../components/buttons.css");  
@import url("../components/cards.css");  
@import url("../components/navbar.css");
```

```
@import url("../components/footer.css");

/* 4. Utility styles */
@import url("../utilities/colors.css");
@import url("../utilities/margins.css");
@import url("../utilities/animations.css");
```

4.2 Example 2: Theme System with @import

[colback=codegray, colframe=darkblue, title=File Structure]

```
css/  
  main.css  
  core/  
    layout.css  
    typography.css  
    components.css  
  themes/  
    light.css  
    dark.css  
    high-contrast.css
```

[colback=codegray, colframe=darkblue, title=main.css]

```
/* Core structure */  
@import url("../core/layout.css");  
@import url("../core/typography.css");  
@import url("../core/components.css");  
  
/* Theme styles based on preference */  
@import url("../themes/light.css");  
  
/* Dark mode theme */  
@media (prefers-color-scheme: dark) {  
  @import url("../themes/dark.css");  
}  
  
/* High contrast mode */  
@media (prefers-contrast: more) {  
  @import url("../themes/high-contrast.css");  
}
```

4.3 Example 3: Google Fonts Organization

[colback=codegray, colframe=darkblue, title=CSS Code]

```
/* fonts.css - Import all fonts at once */  
@import url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&  
  family=Playfair+Display:wght@700&family=Open+Sans:wght@300;400&display=  
  swap');  
  
/* variables.css - Define font variables */  
:root {  
  --font-primary: 'Roboto', sans-serif;  
  --font-display: 'Playfair Display', serif;  
  --font-secondary: 'Open Sans', sans-serif;  
}  
  
/* typography.css - Apply fonts */  
body {  
  font-family: var(--font-primary);
```

```
}  
  
h1, h2, h3 {  
    font-family: var(--font-display);  
}  
  
.subtitle {  
    font-family: var(--font-secondary);  
}
```

Chapter 5

Best Practices

5.1 Practice 1: Limit Import Depth

Avoid importing files that import other files (deep nesting):

[colback=codegray, colframe=darkblue, title=GOOD - Flat Structure]

```
/* main.css */
@import url("./reset.css");
@import url("./components.css");
@import url("./utilities.css");
```

[colback=codegray, colframe=darkblue, title=AVOID - Deep Nesting]

```
/* main.css imports... */
@import url("./layers/base.css");

/* layers/base.css imports... */
@import url("./core/reset.css");

/* This creates unnecessary HTTP requests */
```

5.2 Practice 2: Use @import in Preprocessors

Modern approach with SASS/LESS:

[colback=codegray, colframe=darkblue, title=SCSS Example]

```
// main.scss
@import 'variables';
@import 'mixins';
@import 'base/reset';
@import 'base/typography';
@import 'components/buttons';
@import 'components/cards';

// SCSS handles compilation to single CSS file
// Results in ONE HTTP request, not multiple
```

5.3 Practice 3: Critical Path Optimization

Inline critical styles, defer non-critical:

[colback=codegray, colframe=darkblue, title=HTML]

```
<!-- Critical styles inline -->
<style>
  body { margin: 0; font-family: sans-serif; }
  header { background: #333; color: white; }
</style>

<!-- Main stylesheet -->
<link rel="stylesheet" href="css/main.css">

<!-- Non-critical styles deferred -->
<link rel="stylesheet" href="css/animations.css" media="print">
```

5.4 Practice 4: Name Imports Meaningfully

Use clear, descriptive filenames:

```
/* GOOD - Clear intent */
@import url("./components/button.css");
@import url("./layout/grid-system.css");
@import url("./utilities/color-palette.css");

/* AVOID - Unclear */
@import url("./style1.css");
@import url("./s2.css");
@import url("./tmp.css");
```

5.5 Practice 5: Organize Import Order

Follow a logical structure:

```
/* 1. Variables and functions */
@import url("./variables.css");
@import url("./mixins.css");

/* 2. Reset and normalize */
@import url("./reset.css");

/* 3. Base styles */
@import url("./typography.css");
@import url("./forms.css");

/* 4. Layout */
@import url("./layout.css");

/* 5. Components */
@import url("./components.css");
```

CSS @import Rule

```
/* 6. Utilities and overrides */  
@import url("../utilities.css");  
  
/* 7. Responsive styles */  
@import url("../responsive.css");
```

Chapter 6

Performance Considerations

6.1 1. HTTP Requests

Each @import creates an additional HTTP request:

[colback=codegray, colframe=darkblue, title=SLOW - Multiple Imports]

```
@import url("./reset.css");
@import url("./buttons.css");
@import url("./cards.css");
@import url("./modals.css");
/* 4+ HTTP requests */
```

[colback=codegray, colframe=darkblue, title=FAST - Single File]

```
/* Use build tools to bundle all imports into one file */
/* OR use <link> tags in HTML for parallel loading */
```

6.2 2. Use Build Tools

Webpack, Parcel, Vite compile imports into single file:

```
// webpack.config.js
module: {
  rules: [{
    test: /\.css$/,
    use: ['style-loader', 'css-loader']
  }]
}

// Import as module
import './styles.css';
```

6.3 3. Critical CSS Inlining

Inline critical styles, defer rest:

```
<!-- Inline critical styles for fast first paint -->
<style>
```

```
/* Only critical styles here */
body { margin: 0; font-family: sans-serif; }
header { background: #333; }
</style>

<!-- Load non-critical styles -->
<link rel="stylesheet" href="css/main.css">
```

6.4 4. Font Loading Strategy

```
/* Use font-display to optimize loading */
@import url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&
  display=swap');

/* display=swap shows fallback font immediately */
/* display=optional prevents layout shift but may not show font */
```

Chapter 7

Real-World Examples

7.1 Example 1: Design System

[colback=codegray, colframe=darkblue, title=Design System Structure]

```
design-system/  
  index.css  
  tokens/  
    colors.css  
    typography.css  
    spacing.css  
  components/  
    button.css  
    input.css  
    card.css  
  utilities/  
    helpers.css  
    animations.css
```

[colback=codegray, colframe=darkblue, title=index.css]

```
/* Design tokens */  
@import url("../tokens/colors.css");  
@import url("../tokens/typography.css");  
@import url("../tokens/spacing.css");  
  
/* Components */  
@import url("../components/button.css");  
@import url("../components/input.css");  
@import url("../components/card.css");  
  
/* Utilities */  
@import url("../utilities/helpers.css");  
@import url("../utilities/animations.css");
```

7.2 Example 2: Multi-Brand Setup

[colback=codegray, colframe=darkblue, title=brands/brand-a.css]

```
/* Shared base styles */
@import url("../base/typography.css");
@import url("../base/layout.css");

/* Brand A specific colors */
@import url("../colors-a.css");
@import url("../components-a.css");
```

[colback=codegray, colframe=darkblue, title=brands/brand-b.css]

```
/* Shared base styles */
@import url("../base/typography.css");
@import url("../base/layout.css");

/* Brand B specific colors */
@import url("../colors-b.css");
@import url("../components-b.css");
```

7.3 Example 3: Framework Integration

[colback=codegray, colframe=darkblue, title=main.css]

```
/* Import framework */
@import url('https://cdn.jsdelivr.net/npm/bootstrap@5/dist/css/bootstrap.min.css');

/* Override with custom variables */
@import url("../variables.css");

/* Add custom components */
@import url("../components/custom-button.css");
@import url("../components/custom-card.css");

/* Custom utilities */
@import url("../utilities/spacing-overrides.css");
```

Chapter 8

Common Mistakes to Avoid

8.1 Mistake 1: @import Not at Start of File

```
/* WRONG - @import after other rules */
body { margin: 0; }
@import url("./reset.css");

/* RIGHT - @import at beginning */
@import url("./reset.css");
body { margin: 0; }
```

8.2 Mistake 2: Circular Imports

```
/* main.css */
@import url("./components.css");

/* components.css */
@import url("./main.css");

/* This creates infinite loop */
```

8.3 Mistake 3: Deep Nesting

```
/* level1.css */
@import url("./level2.css");

/* level2.css */
@import url("./level3.css");

/* level3.css */
@import url("./level4.css");

/* Creates unnecessary HTTP requests */
```

8.4 Mistake 4: Wrong File Paths

```
/* WRONG - Incorrect path */
@import url("styles/components.css");

/* RIGHT - Correct relative path */
@import url("../styles/components.css");

/* RIGHT - Absolute URL */
@import url("https://example.com/styles.css");
```

8.5 Mistake 5: Performance Ignored

```
/* WRONG - Too many individual imports */
@import url("../btn.css");
@import url("../card.css");
@import url("../modal.css");
/* ... 20+ more imports */

/* RIGHT - Use build tools or limit imports */
/* Bundle into one file with Webpack, etc. */
```

Chapter 9

Tips and Best Practices

9.1 Tip 1: Use CSS Preprocessors

SASS/LESS handle imports better:

```
// styles.scss
@import 'variables';
@import 'mixins';
@import 'base/reset';
@import 'components/buttons';

// Compiles to single CSS file
// No performance penalty from multiple imports
```

9.2 Tip 2: Leverage Build Tools

Modern tooling optimizes imports:

```
// Modern setup with Vite, Next.js, etc
import './styles.css';
import './components.css';

// Tools bundle and optimize automatically
```

9.3 Tip 3: Document Your Structure

```
/**
 * Main stylesheet
 *
 * Organization:
 * 1. Variables - Design tokens
 * 2. Base - HTML element styles
 * 3. Layout - Grid and flex
 * 4. Components - Reusable parts
 * 5. Utilities - Helper classes
 * 6. Responsive - Media queries
 */
```

```
@import url("../variables.css");
@import url("../base.css");
@import url("../layout.css");
```

9.4 Tip 4: Use Meaningful Naming

```
/* GOOD */
@import url("../components/button-primary.css");
@import url("../layout/grid-12col.css");

/* AVOID */
@import url("../style.css");
@import url("../s1.css");
```

9.5 Tip 5: Monitor Performance

Test your import strategy:

```
/* Use DevTools to check:
 * - Number of HTTP requests
 * - CSS file sizes
 * - Load times
 * - Parse times
 */
```

Chapter 10

Summary

[colback=lightblue, colframe=darkblue, title=What You Learned]

Key Points About @import:

1. Imports external CSS files into stylesheet
2. Must be placed at beginning of file
3. Supports local files and URLs
4. Can include media queries and supports conditions
5. Enables modular CSS organization
6. Each import creates HTTP request
7. Best used with build tools
8. Alternative to HTML `<link>` tags

Common use cases:

- Importing Google Fonts
- Loading framework stylesheets
- Organizing modular CSS
- Importing vendor stylesheets
- Theme management
- Design system distribution
- Component library usage
- Conditional stylesheet loading

Master @import to create well-organized, maintainable CSS projects!

Chapter 11

Practice Exercises

11.1 Exercise 1: Modular Architecture

Create a CSS project with:

- Separate reset file
- Variables file
- Components file
- Utilities file

11.2 Exercise 2: Theme System

Build a theme system that:

- Has light and dark themes
- Imports conditionally
- Uses CSS variables

11.3 Exercise 3: Font Organization

Create a fonts.css that:

- Imports multiple Google Fonts
- Defines font variables
- Uses fonts across styles

11.4 Exercise 4: Framework Integration

Integrate Bootstrap using @import:

- Import Bootstrap CSS
- Override with custom variables
- Add custom components

11.5 Exercise 5: Multi-File Project

Organize a project into:

- Base styles
- Layout styles
- Component styles
- Utility styles
- Responsive styles

11.6 Exercise 6: Performance Optimization

Optimize CSS imports by:

- Using build tools
- Combining files
- Measuring performance

Complete these exercises to master CSS @import!